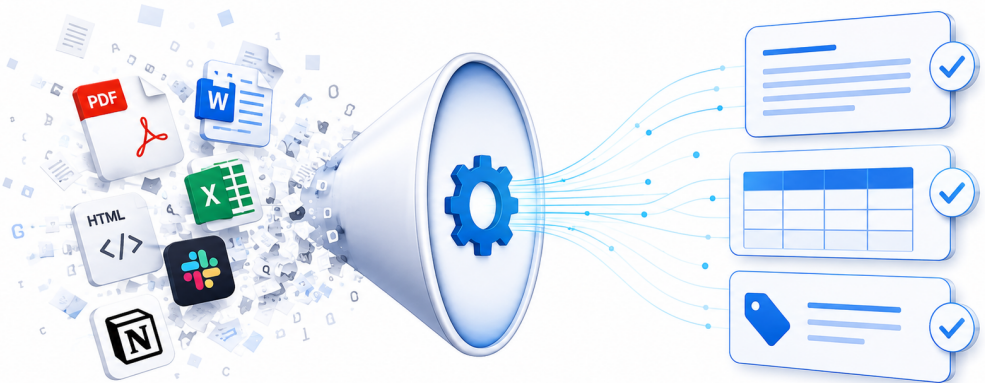


Data Cleaning FOR RAG

Nobody posts about this. It is the single biggest lever on whether your **RAG pipeline actually works.**



Why Data Determines Quality

Retrieval can only find what you put in the index

REPORTED SYMPTOM	ROOT CAUSE UPSTREAM
"the model hallucinated"	right chunk was never retrieved
"retrieval is bad"	answer split across two chunks
"cites the wrong source"	same content exists 6x, never deduped
"scores are all ~0.7"	every chunk has the same boilerplate
"says 'I don't know'"	PDF extractor dropped the tables

No embedding model, reranker, or prompt fixes any of these. They are all **upstream data defects**.

🔗 THE UNCOMFORTABLE RATIO

Data preparation is the **majority of the work and the quality gain**. Model selection is a rounding error next to it.



The Full Data Pipeline

Seven stages, and cleaning is most of them

1. **SOURCE** · PDFs, HTML, Word, Notion, Slack



2. **EXTRACT** · Get text out of the container

INFO DIES HERE

3. **CLEAN** · Encoding, whitespace, boilerplate



4. **STRUCTURE** · Headings, sections, tables, lists



5. **DEDUPLICATE** · Exact, near, semantic



6. **ENRICH** · Metadata, titles, dates, permissions



7. **CHUNK** · Split along structure, not char counts



Stages 2 to 6 are what this carousel is about. Most tutorials jump straight from 1 to 7.



AI Coach John

Follow



PROITBRIDGE
Solve For Better Future



Repost

Where Data Silently Dies

The text you extract is not the text on the page

📄 MULTI-COLUMN PAGE

```
[Col 1: Revenue grew]
[Col 2: Costs fell]
```

← What a human reads on the page

✖ NAIVE EXTRACTION

```
"Revenue grew Costs fell in
2024 due to sharply due ..."
```

← Columns interleaved into nonsense

📊 TABLE GRID ON PAGE

Region	Q1	Q2
EMEA	4.2	5.1

← Rows and columns carry the meaning

✖ FLATTENED TABLE

```
"Region Q1 Q2 EMEA 4.2 5.1
APAC 3.3 3.9"
```

← Row/column relationships destroyed

Why it matters: interleaved text embeds to a meaningless vector. It is never retrieved for the right question, and when it is, the model reads gibberish.



Choosing an Extractor

Match the tool to the document, not to habit

1. Simple, text-based PDFs

```
from pypdf import PdfReader
text = "\n".join(p.extract_text() for p in PdfReader(path).pages)
```

2. Multi-column, layout aware

```
import fitz
blocks = fitz.open(path)[0].get_text("blocks")
```

3. Real tables

```
import pdfplumber
tables = pdfplumber.open(path).pages[0].extract_tables()
```

4. Scanned pages, OCR pipeline

```
import pytesseract, pdf2image
text =
pytesseract.image_to_string(pdf2image.convert_from_path(path)[0])
```

Test: does `extract_text()` return real text?



NO → scanned, use OCR

YES → check the layout



Rule: print page 1 of 5 representative docs. Five minutes of reading beats a week of debugging retrieval.



AI Coach John

Follow



PROITBRIDGE
Solutions for Smarter Teams



Repost

Encoding & Glyph Cleanup

Invisible characters that break exact matching

```
import re, unicodedata

LIGATURES = {"fi": "fi", "fl": "fl", "ffi": "ffi"}

def normalise_text(text: str) → str:
    # 1. Unicode normalisation
    text = unicodedata.normalize("NFKC", text)
    # 2. Replace PDF ligature glyphs
    for lig, repl in LIGATURES.items():
        text = text.replace(lig, repl)
    # 3. Strip zero-width and non-breaking spaces
    text = text.replace(" ", "").replace("\xa0", " ")
    return text
```

Q WHY THIS IS NOT COSMETIC

User searches "financial reporting". The chunk holds "financial reporting" as one ligature glyph.

- **BM25 keyword search:** **no match at all**
- **Embedding search:** weaker match, tokenised differently



Normalise once, at ingest. Doing it at query time cannot repair what is already in the index.



AI Coach John

Follow



PROITBRIDGE
Solutions For Better Futures



Repost

Whitespace & Hyphenation

Two fixes that measurably improve retrieval

```
def fix_whitespace(text: str) → str:
    # 1. Join words broken across a line break
    text = re.sub(r"(\w+)-\s*\n\s*(\w+)", r"\1\2", text)
    # 2. Single newline inside a paragraph → space
    text = re.sub(r"(?!\\n)\\n(?!\\n)", " ", text)
    # 3. Collapse blank-line runs to one break
    text = re.sub(r"\\n{3,}", "\\n\\n", text)
    return text.strip()
```

✗ BEFORE

"met all require-
ments for reporting."

✓ AFTER

"met all requirements
for reporting."



Underrated: left alone, "require-ments" never matches "requirements" in keyword or vector search.



Boilerplate Removal

The repeated text that poisons every chunk

```
from collections import Counter

def find_boilerplate(pages, threshold=0.6) → set[str]:
    # Lines on >60% of pages are headers/footers
    counts = Counter(
        line.strip()
        for p in pages
        for line in set(p.split("\n"))
    )
    cutoff = len(pages) * threshold
    return {l for l, c in counts.items()
            if c ≥ cutoff and len(l) > 3}
```

🔪 WHY BOILERPLATE POISONS CHUNKS

Every chunk opens with "Acme Corp Confidential | Page N of 240".

- **Shared signal:** all chunks look alike to the embedder
- **Score compression:** similarity collapses toward ~0.7 for every query

HTML sources: run `trafilatura` or `readability-lxml` to strip nav, sidebars and footers before you ever see the text.



Headings as Context

A chunk without its heading path has lost its meaning

✗ FLAT CHUNK

"...must be submitted within 30 days of the qualifying event."

← No clue which policy this is

✓ STRUCTURED CHUNK

Handbook > Benefits > Life Events

"...must be submitted within 30 days..."

← Retrievable and citable

```
def build_heading_path(blocks) → list[dict]:
    path, out = {}, []
    for b in blocks:
        if b["type"] == "heading":
            path[b["level"]] = b["text"]
        else:
            trail = " > ".join(path[k] for k in sorted(path))
            out.append(**b, "heading_path": trail)
    return out
```



Highest return move: prepend the heading path to each chunk before embedding.



Handling Tables

The content type naive pipelines destroy completely

× **FLATTENED** "Plan Storage Price Pro 100GB \$29"

DESTROYS ANSWERS

✓ **MARKDOWN** | Plan | Storage | Price |

PRESERVES GRID

✓✓ **ROW SENTENCES** "Pro includes 100GB for \$29/mo"

RETRIEVABLE

```
def table_to_sentences(headers, rows, caption=""):
    # One independently retrievable sentence per row
    for r in rows:
        facts = ", ".join(f"{h} is {v}"
                          for h, v in zip(headers, r))
        yield f"{caption} {facts}".strip()
```

Rule of thumb: small tables stay as markdown inside the chunk, large tables become row-level sentences. Never flatten into a space-separated string.



Deduplication Strategies

Three kinds of duplicate, three different fixes

1. **EXACT** · normalise, then SHA-256 hash

CHEAP

2. **NEAR** · shingle overlap, Jaccard > 0.9

MEDIUM

3. **SEMANTIC** · cosine of embeddings > 0.95

COSTLY

```
import hashlib, numpy as np

# 1. Exact duplicate
norm = re.sub(r"\s+", " ", text.strip().lower())
h = hashlib.sha256(norm.encode()).hexdigest()

# 2. Near duplicate
jaccard = len(sa & sb) / len(sa | sb)

# 3. Semantic duplicate
keep = [i for i, e in enumerate(embs)
        if all(np.dot(e, embs[j]) < 0.95 for j in keep)]
```

! IMPACT ON RETRIEVAL

Without dedupe, `top_k = 5` returns five copies of the same refund policy (v2, v3, FAQ, email thread). **Every diverse slot is wasted.**



Versioning & Conflicts

Deduplication's harder sibling

⚠️ CONTRADICTION HAZARD

```
policy_v2.pdf : "Refunds within 14 days."  
policy_v3.pdf : "Refunds within 30 days."
```

Both indexed → the model picks one at random, or blends them into an answer that was never true.

1. INDEX CURRENT ONLY · superseded docs never enter

2. METADATA FILTER · filter={"status": "current"}

3. RECENCY RANKING · boost effective_date in rerank



Governance question: who marks a document superseded? If the answer is "nobody", the index drifts into contradiction.



AI Coach John

Follow



PROITBRIDGE
Solutions for Better Futures



Repost

Metadata **Enrichment**

Fields that make retrieval filterable and traceable

```
from dataclasses import dataclass

@dataclass
class ChunkMetadata:
    # 1. Provenance: citations and debugging
    source_id: str; source_title: str
    page_number: int; heading_path: str
    # 2. Filtering: hybrid precision search
    doc_type: str; department: str
    effective_date: str; status: str
    # 3. Access control: the security layer
    allowed_roles: list[str]; tenant_id: str
    # 4. Derived: monitoring
    token_count: int; chunk_index: int
```



PROVENANCE

Cite sources and debug wrong answers back to a page.



SECURITY

Enforce access at the data layer, never in the prompt.



Chunk Quality Checks

Validate chunks before they ever reach the index

```
def validate_chunk(text: str, meta: dict) → list[str]:
    problems = []
    tokens = count_tokens(text)
    if tokens < 20:
        problems.append("too_short")
    if tokens > MODEL_LIMIT:
        problems.append("exceeds_embedding_limit")
    if alnum_ratio(text) < 0.6:
        problems.append("mostly_symbols")
    if not re.search(r"[.!?]", text):
        problems.append("no_sentence_boundary")
    return problems
```

🚩 THE SILENT TRUNCATION TRAP

exceeds_embedding_limit is the dangerous one. The model truncates without warning: feed a 900-token chunk to a 512-token embedder and the last 400 tokens were never embedded at all.



Log every rejection. A spike in one problem type points straight at the source format that broke.



Structural Chunking

Split at meaning boundaries, not character counts

✗ FIXED-SIZE SPLIT

Cuts mid-sentence and merges two unrelated policies into one chunk.

✓ STRUCTURE-AWARE

Keeps the section whole and prepends its heading path.

```
def structure_aware_chunks(sections, max_tokens=500):  
    for sec in sections:  
        prefix = f"{sec['heading_path']}\n\n"  
        if count_tokens(prefix + sec["text"]) ≤ max_tokens:  
            yield prefix + sec["text"]  
        else:  
            budget = max_tokens - count_tokens(prefix)  
            for p in split_paragraphs(sec["text"], budget):  
                yield prefix + p
```



Sections stay whole whenever they fit, and every chunk carries its heading path.



Keeping the Index Fresh

Cleaning is a pipeline, not a one-time script

```
def sync_document(doc_id: str, content: bytes, store) → str:
    h = hashlib.sha256(content).hexdigest()
    if store.get_meta(doc_id).get("content_hash") == h:
        return "unchanged"

    # Delete old chunks BEFORE writing new ones
    store.delete_chunks(filter={"source_id": doc_id})

    chunks = clean_and_chunk(content)
    store.upsert(chunks, doc_id=doc_id, content_hash=h)
    return "reindexed"
```

UPDATE · delete old chunks before adding new

ORPHANS

DELETE · propagate source deletes to the index

LEAKS

REPROCESS · rebuild the corpus on model change

DRIFT



A content hash per document turns a full nightly rebuild into **seconds of incremental work**.



AI Coach John

Follow



PROITBRIDGE
Solutions for better future



Repost

Proving Cleaning Helped

Measure retrieval, not vibes

```
def evaluate_retrieval(eval_set, retriever, k=5) → dict:
    hits, rr = 0, []
    for case in eval_set:
        got = [d.id for d in retriever.search(case["q"], k=k)]
        want = set(case["expected_ids"])
        if want & set(got):
            hits += 1
            rank = next(i for i, d in enumerate(got)
                        if d in want)
            rr.append(1 / (rank + 1))
    return {"hit_rate": hits / len(eval_set),
            "mrr": sum(rr) / len(eval_set)}
```

1. Build a 30-50 question eval set

ONCE

2. Measure the baseline, then change ONE step

ISOLATE

3. Re-measure, then keep it or revert it

DECIDE



No LLM in the loop: retrieval metrics are cheap, fast and deterministic.



AI Coach John

Follow



PROITBRIDGE
Solve For Better Future



Repost

Common RAG Mistakes

What quietly breaks a RAG corpus

REPORTED SYMPTOM	LIKELY DATA CAUSE
"answer is half right"	answer split across a chunk boundary
"similarity scores ~0.7"	boilerplate on every chunk
"top_k is all the same"	no deduplication
"cites outdated policy"	old chunks never deleted
"the numbers are wrong"	table flattened during extraction
"works for some docs"	one format extracting badly
"worse after upgrade"	embedding changed, corpus not rebuilt



Every row is a data bug wearing a model bug's clothes. Check the corpus first.



AI Coach John

Follow



Repost

Pipeline Checklist

Run this before you blame the model

📄 EXTRACTION

- ✓ Read 5 docs by hand
- ✓ Column order correct
- ✓ Tables preserved
- ✓ OCR for scans

👤 STRUCTURE & META

- ✓ Heading path prepended
- ✓ Provenance fields
- ✓ Access control enforced
- ✓ Tables kept as rows

✂️ CLEANING

- ✓ Unicode normalised
- ✓ Line hyphens joined
- ✓ Headers stripped
- ✓ Ligatures replaced

🔄 FRESHNESS & DEDUPE

- ✓ Near-dedupe active
- ✓ Updates delete old
- ✓ Baseline metrics saved
- ✓ Reindex on model swap



Sixteen boxes. Any unticked box is a cheaper fix than swapping your embedding model.



AI Coach John

Follow



PROITBRIDGE
Solve For Better Future



Repost

The model can only be as good as the corpus

Most "model" failures are extraction, boilerplate, structure or duplication problems wearing a disguise.

1. **Extract carefully** · do not lose info at the door

2. **Normalise text** · make matching possible

3. **Strip boilerplate** · stop chunks looking alike

4. **Keep structure** · give chunks their context back

5. **Deduplicate** · make every top_k slot count

6. **Enrich metadata** · enable filtering and citation

7. **Validate chunks** · catch silent truncation

8. **Measure retrieval** · prove any of it worked



Found this useful? **Follow AI Coach John** for more production AI engineering.



AI Coach John

Follow



PROITBRIDGE
Solutions for Better Futures



Repost